

V3 Tenancy & NATS Security Model

Design review — Linebooker V2 vs. the NATS Dictionary POC, and the model to take forward

Working session transcript · Claude Code (Opus 5) · repository `nats-tech-lab`, branch `poc/dictionary1.24` · 9–11 August 2026

JEREMY

[Attached: `logistics-nats-linebooker-chat.md` — a prior ChatGPT conversation] First confirm you're able to read my previous chat from ChatGPT.

The file was readable once moved out of `~/Downloads` (macOS TCC blocks that folder for this process, so the first attempt failed with `Operation not permitted` despite the file being visible). It is a two-turn conversation:

Turn 1 — modelling Admin / Operator / Customer / Transporter. Platform Admin is not a tenant (AWS-root analogy); Operator is the legal/business entity per territory and maps to a NATS Account; Customer and Transporter are children of the Operator. Users belong to an Organisation, not to roles directly — Operator, Customer and Transporter are all specialisations of a generic `Organisation`.

Turn 2 — reworked for a Linebooker-style marketplace. Reverses the ownership assumption: Customers and Transporters are independent organisations participating in an Operator's marketplace, joined via a many-to-many `OperatorMembership`. NATS accounts become `$SYS`, `PLATFORM`, `ZA`, `NA`, `AU` — explicitly no `za`. token and no org id in subjects, with org identity in the message body. Per-org isolation via user JWT permissions inside one account, splitting internal domain subjects (`load.created`) from external org-facing subjects (`org.<orgId>.load.*`). No direct Customer→Transporter messaging. A small `PLATFORM` account exports to regional accounts. Final refinement: *NATS Account = isolation boundary*, not automatically = tenant.

JEREMY

Yeah, I'm aware that this might conflict with the current POC design. The design will eventually align with the previous Linebooker V2 code repo, with the product site being `linebooker.com`.

I'm at this stage trying to find the best way to define the model (both business and NATS security) in terms of top roles (admin, operator, company, transporter, user) because the goal is to have a multi-tenant + multi-region distributed V3 Linebooker solution. The core logistics logic will be retained in terms of tenders and bids, loads, payments, etc.

The POC was primarily done to learn/check/confirm how NATS can enable: multi-tenancy, data isolation and security, event sourcing, etc.

The main goal: Please review Linebooker V2 and the POC at a high level and confirm if the ChatGPT snapshot model design is the way to go. You can also recommend a better alternative, if one exists.

Verdict

The ChatGPT model is roughly 70% right, and the POC already implements a better version of several parts — but it contains one structural error that would cost a lot to unwind later, and it leaves the most important question (isolation *between organisations*) under-specified.

What V2 actually is — three findings that change the answer

1. **V2 already has the generic-organisation model the chat recommends.** `BusinessEntity` + `BusinessType {CUSTOMER, TRANSPORTER, OPERATOR, INTEGRATOR}`, with `CustomerProfileEntity` / `TransporterProfileEntity` as optional 1:1 role profiles. The chat presents "Organisation with specialisations" as a new idea; it shipped years ago. Keep it.
2. **V2 is not multi-tenant at all** — one MySQL schema `linebooker_v2`, zero tenant discriminators, no Hibernate multi-tenancy. Isolation today is environment-per-deployment: `qa/uat/dm/thornlands/prd` profiles, and critically a dedicated white-label build pipeline (`infra/cloud-build/new-stack-no-db-whitelabel.yaml`). That white-label stack is the single most important piece of evidence in the whole review.
3. **In V2, "Operator" means Linebooker itself** (`BusinessType.OPERATOR` = platform staff), and **"Region" means a province inside a country** (`RegionEntity` ; `country → region → town` ; `transporter_profile_entity_region` for coverage). The chat uses both words to mean something completely different.

The marketplace intermediation the chat argues for is already real in V2: `BusinessVisibilityEntity` (bidirectional INCLUDE/EXCLUDE lists), `CustomerTransporterAllocationEntity` (the allocated-transporter panel), rate-sheet entries keyed by both customer-route and transporter. Customers and transporters already never talk directly.

Where the chat is right — adopt these

- **Organisation as a generic entity with a type discriminator.** Matches V2; extends to warehouses/brokers/ports without schema churn.
- **Membership is many-to-many.** Woolworths participating in ZA and NA must not be two Woolworths rows. V2 has no equivalent (a `UserEntity` belongs to exactly one `BusinessEntity`) precisely because V2 has exactly one marketplace. A genuine gap, correctly named.
- **Tenant identity never goes in the subject** — the account *is* the namespace. The POC already enforces this.
- **No direct customer→transporter messaging;** route everything through marketplace services so eligibility, lanes, pricing, blacklists and tender rules stay enforceable. This is V2's actual behaviour, and it is correct.
- **A small PLATFORM account that exports to regional/tenant accounts.** The POC built exactly this in Phase 21. It works; don't redesign it.
- **"NATS Account = isolation boundary, not a mirror of the domain hierarchy."** The chat's closing refinement is the right principle — it just doesn't follow its own advice in the section above it.

The structural error: account = region

The chat lands on `NATS Account ∈ {PLATFORM, ZA, NA, AU}` — one account per regional marketplace. That collapses **two independent axes into one slot**, and the POC deliberately separated them in Phase 16a: *region = deployment, tenant = NATS account, company/BU = {context}* .

- **Region is a deployment / data-residency axis.** ZA and AU are different NATS servers, different Postgres, different latency and legal domains. Cross-region movement is JetStream replication, not subject routing. A NATS account is a within-server construct — it cannot express "different continent."
- **If account == region, there is no hard isolation left inside a market.** All 500 customers and 300 transporters in ZA share one account: one JetStream resource pool, one KV namespace, one blast radius. Every isolation guarantee then rests on subject-permission strings in JWTs rather than the server-enforced account wall the POC proved out.

- **The infrastructure says the axes must stay separate.** There is already a white-label build pipeline and a named non-Linebooker stack (`thornlands`). A tenant is "a business operating a marketplace" — not a geography. Linebooker-ZA and a white-label operator can perfectly well live in the same ZA deployment; under the chat's model they'd share an account.

Correction: Tenant = the marketplace-operating business, not the region. Region is which stack you are deployed on. If Linebooker only ever runs one marketplace per country, the two collapse and the distinction is free — which is exactly why it should be made now rather than after the first white-label customer.

The under-specified part: isolating organisations inside an account

The chat wobbles here, and it is the decision that actually matters. It first says org identity belongs in the message body, not the subject — then, three sections later, introduces `org.<orgId>.>` subjects because a browser on a WebSocket needs subject-level scoping. The second position is correct and the first is dangerous: **if org identity is only in the payload, NATS can enforce nothing**, and you are back to application-level authz on every message with a browser holding `SUB >` .

The subtlety the chat misses: within a single account, a responding service **does not receive the caller's identity** from NATS (`Nats-Request-Info` only exists on cross-account service imports). So per-organisation authorisation has to be subject-encoded to be server-enforced, in *both* directions — not just on the fan-out side. That points at a two-plane subject design:

Plane	Subjects	Who reaches it
Backend	<code>evt.{context}.loads.load.{id}.created</code> <code>rpc.{context}.pricing.rate.get.v1</code>	Services only — marketplace-wide, no browser access
Participant	<code>api.{context}.</code> <code>{org}.loads.load.create.v1</code> <code>org.{context}.{org}.tender.{id}.offered</code>	Browsers — JWT allows only its own <code>{org}</code>

A load event stays on the unscoped backend plane (many transporters must see it); a matching/tender service decides eligibility and fans out onto the participant plane. That is the chat's internal/external split, made enforceable rather than advisory.

Two implications for the POC as it stands. Today `MintBrowserToken` grants `Pub api.>` / `Sub api.>`, `notify.>`, `obs.api.>` — wide open inside the account. Fine when the account holds one company; not fine when it holds 800 competing businesses. And **the subject grammar is the one thing to decide**: `domain.SubjectDetails` currently rejects anything that is not exactly 6 tokens, so adding an `{org}` token breaks it. Either accept arity 7 for `api.*` , or give the participant plane its own family with its own fixed arity — I would pick the latter, so the backend parsers stay untouched and the two planes stay visibly distinct.

Recommended model

#	Axis	Meaning	Enforcement
1	Region	Deployment / data residency (za, au)	Separate stack — never in a subject or JWT
2	Tenant	Marketplace-operating business	NATS account — hard, server-enforced
3	Organisation	Customer Transporter Operator-self Integrator (Warehouse, Broker...)	Subject-scoped inside the account
4	Context / BU	Company + business-unit scoping	{context} token — naming scope, not a boundary
5	User	Member of one or more organisations	Credential minted per membership

With membership as a first-class join — Organisation —< OrganisationTenantMembership >— Tenant (status, capabilities, joined_at) and User —< Membership >— Organisation (role[], status). Mapping V2 forward:

V2	V3
BusinessEntity + BusinessType	Organisation + OrganisationType — keep as-is
CustomerProfileEntity / TransporterProfileEntity	Keep the role-profile pattern
BusinessVisibilityEntity , CustomerTransporterAllocationEntity	This <i>is</i> the marketplace eligibility layer — do not reinvent it
UserEntity.business_id + scalar jhi_role	Replace with Membership(user, org, role[])
RegionEntity (province)	Migrate onto GeoAreaEntity and free the word "region"
Environment-per-stack, white-label pipeline	Becomes tenant = NATS account

One extra note on the user model: mint the NATS credential **per active membership**, not per user. A session picks one organisation, gets a JWT scoped to that org, expires in 15–30 minutes (the POC's BR-AC20 / BR-AC21 envelope already does the TTL half). Switching org = re-mint. That gives multi-org users for free and keeps every credential narrow.

Naming hazards — settle before writing V3 code

"Operator" is overloaded three ways: the NATS operator (root of the trust chain), BusinessType.OPERATOR (Linebooker staff in V2), and the chat's regional tenant. Reserve it for the NATS operator and call the business thing **Tenant**. Likewise **"Region"** means a province in V2 and a deployment in the chat — keep it for the deployment axis and move geography onto GeoArea.

Gaps in the POC to close

- GET /api/auth/connectInfo is **unauthenticated** and returns the NKey **seed** to the browser.
- Provisioner.CreateUser mints backend users with **no permissions set at all** — unrestricted within the account. Phase 24b's CreateRestrictedUser is the fix and is still unticked.

- Account signing-key seeds sit in Postgres in plaintext; signing keys accumulate and never rotate.
- `refdata.contexts.tenant` is explicitly *not* a security boundary — fine while `refdata` is global reference data, but per-tenant dictionary customisation cannot live in that column.
- Region and the user/role model (`BR-UA01` – `UA09` , WorkOS) are documented but have **zero code**, and no plan phase covers them.

The three decisions

1. **Is a tenant a marketplace-operating business, or a country?** The white-label pipeline argues for the former — but the commercial roadmap decides, and it is the hinge for everything else.
2. **Do organisations get subject-scoped isolation inside a tenant account from day one, or is that deferred?** Deferring is defensible for a POC; it is not defensible once two competing transporters are on the same account.
3. **Participant-plane subject grammar** — arity-7 `api.*` , or a separate `org.*` family. Prefer the separate family.

JEREMY

Visually map what you're describing showing the business and NATS security model hierarchy.

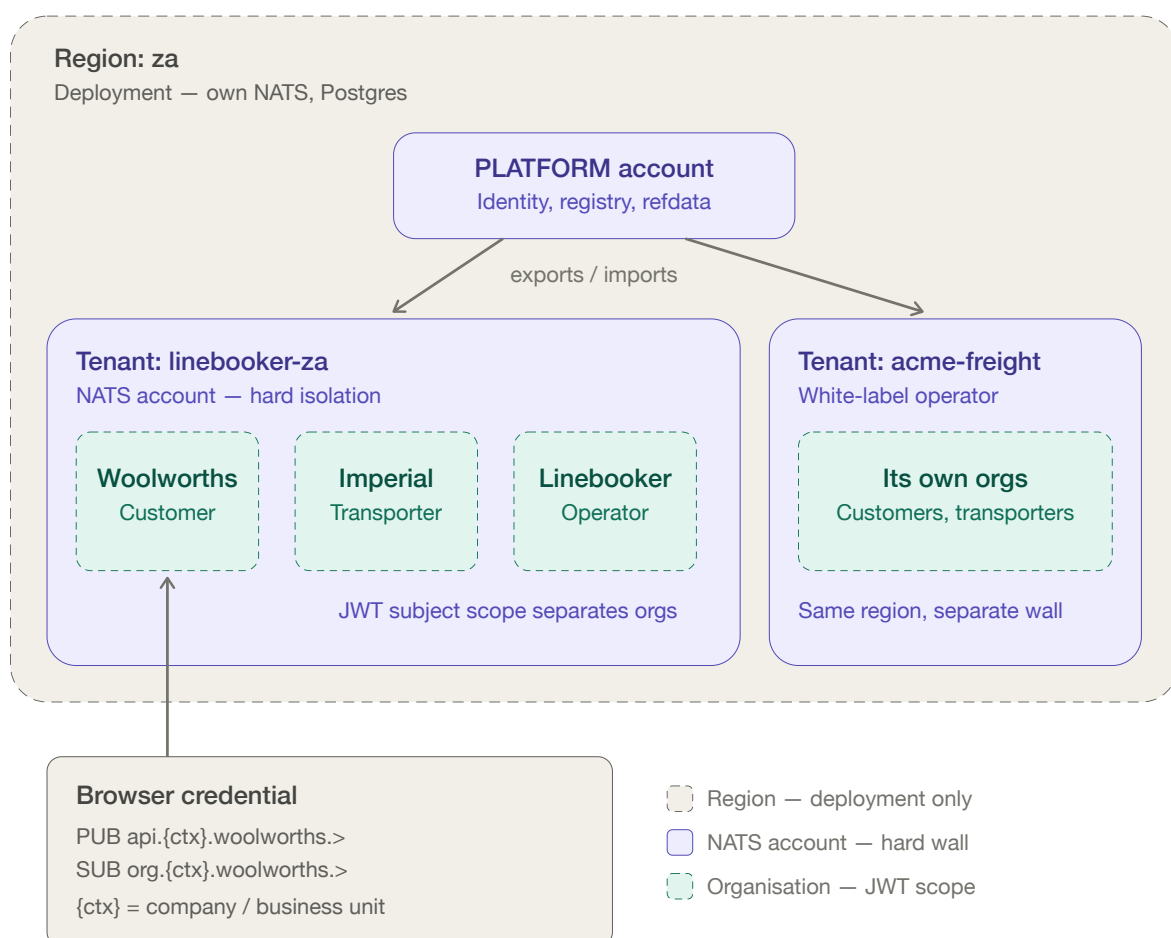


Figure 1 — Single region: the hard wall is the account; organisations inside it are separated by JWT subject scope.

The region is dashed because it isn't a NATS construct at all. It's a deployment: its own server, its own Postgres, its own JetStream store. Nothing in a subject or a JWT ever names it. This is the axis the ChatGPT model collapsed into the account.

The solid purple walls are the only server-enforced boundaries. Two tenants sitting in the same region proves the separation — `linebooker-za` and a white-label operator share infrastructure and share nothing else. Cross-account traffic exists only where PLATFORM explicitly exports and a tenant explicitly imports.

The dashed teal boxes are the soft wall, and that's the decision in front of you. Woolworths and Imperial are competitors inside one account. Nothing at the server level stops one from subscribing to the other's traffic — only the subject allow-list baked into their credential does. That is a legitimate trade (accounts are heavyweight; you don't want 800 of them), but it means the credential-minting path in `auth/token.go` becomes load-bearing security rather than convenience.

The credential is minted per membership, not per user. A person at a 3PL working for two organisations gets two credentials, one per session, each narrow. That is also why `{ctx}` shows up inside the subjects rather than as its own box — it is a naming scope for company/business-unit data, not a security boundary, and it sits *below* the org in the subject.

The escape hatch isn't drawn but follows naturally: if a large customer later demands hard isolation, it graduates from a dashed teal box to its own solid purple one in the same region, joined back to the marketplace by export/import.

JEREMY

Can you show another region deployment in Australia using Linebooker Australia as a tenant.

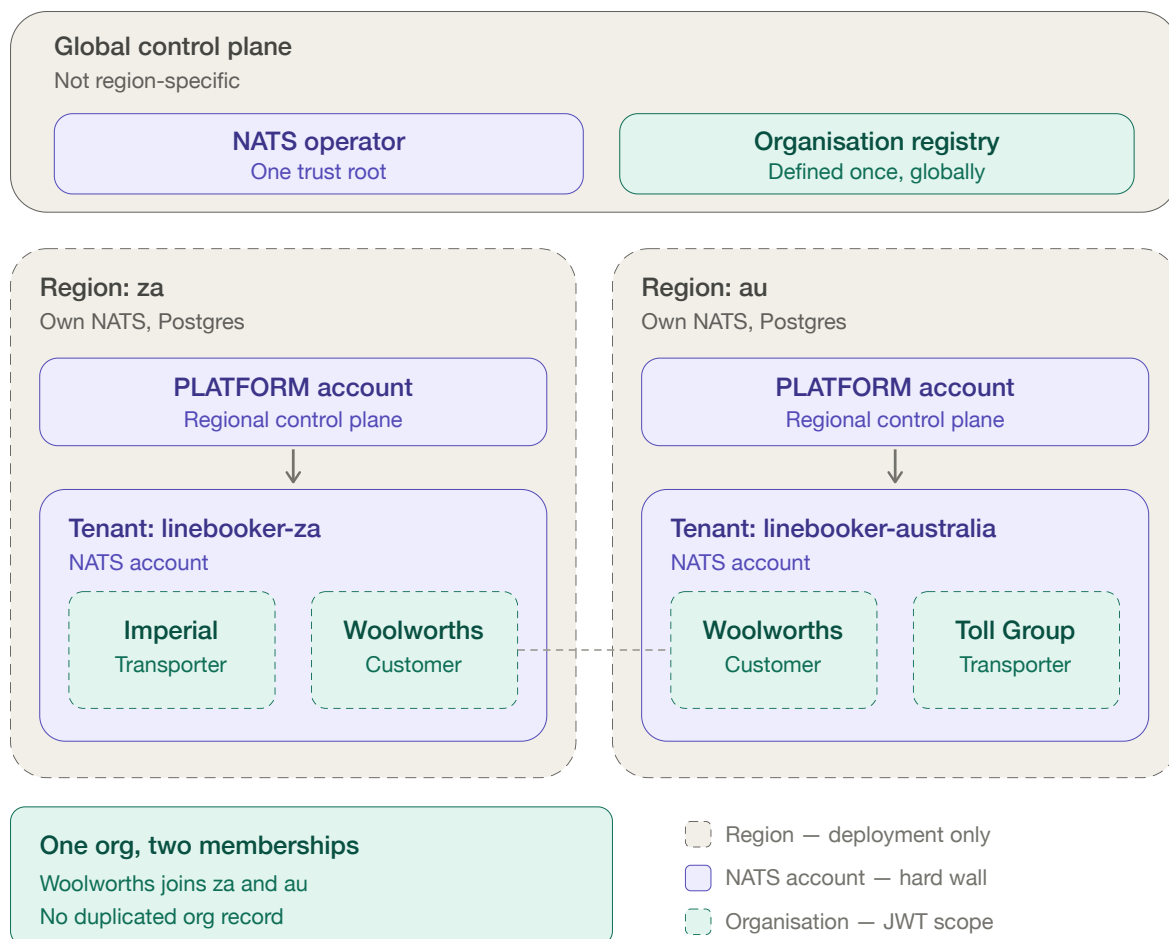


Figure 2 — Two regions. Nothing crosses between them at the NATS level; the dashed link is an identity relationship, not a message path.

Nothing crosses between the regions. There is no gateway, no leaf node, no supercluster, and that is deliberate. The two PLATFORM accounts are siblings, not a hierarchy — `linebooker-za` and `linebooker-australia` cannot see each other's subjects at any price, because they are on different servers. If AU ever needs ZA data, it is a JetStream mirror with an explicit, auditable replication contract, not a subject that happens to resolve. Drawing this out is what shows why "account = region" was the wrong collapse: the account wall would be doing a job the network already does for free, and you would have spent your only isolation primitive on it.

The operator being global is a real decision, not a drawing convenience. One trust root means a tenant's account JWT is cryptographically valid in any region — each regional resolver only has to hold it — so you can stand up `au`, or migrate a tenant between regions, without reissuing anything. The alternative (an operator per region) contains blast radius better but permanently welds every tenant to the region it was born in. Take the single operator with per-region resolvers, and keep the operator signing key offline rather than in a service.

The dashed line between the two Woolworths boxes is not a message path. It is an identity relationship resolved in the registry, in Postgres — NATS knows nothing about it. Woolworths-in-ZA and Woolworths-in-AU are two memberships of one organisation; their loads, tenders and events never mix, and no credential spans both. A Woolworths user working in Australia authenticates against the AU stack and gets an AU-scoped credential. Sessions don't roam.

That last point is where the hard constraint lives. A globally-replicated organisation registry can only carry *identity* — legal name, registration number, which tenants it has joined. The moment operational data (loads,

rates, PODs) gets into it, you have built a cross-border data path that Australian residency rules will eventually object to. Worth writing that boundary down as a rule now, because it is the kind of thing violated by a well-meaning "just add a column."

Two smaller notes: the tenant names in the diagram are inconsistent — `linebooker-za` versus `linebooker-australia`. Tenant names are opaque identifiers that end up in `.creds` filenames and account lookups, so pick one convention (ISO codes throughout) before anything is seeded. And none of the second-region half of this exists yet — the POC is a single NATS server with no regional stack at all, and `Multi-Region-Plan.md` is still marked DRAFT.

JEREMY

Can you also note visually how it relates to NATS account types and JWTs: OPERATOR, ACCOUNT, USER. (docs.nats.io/learn/security/operator-mode)

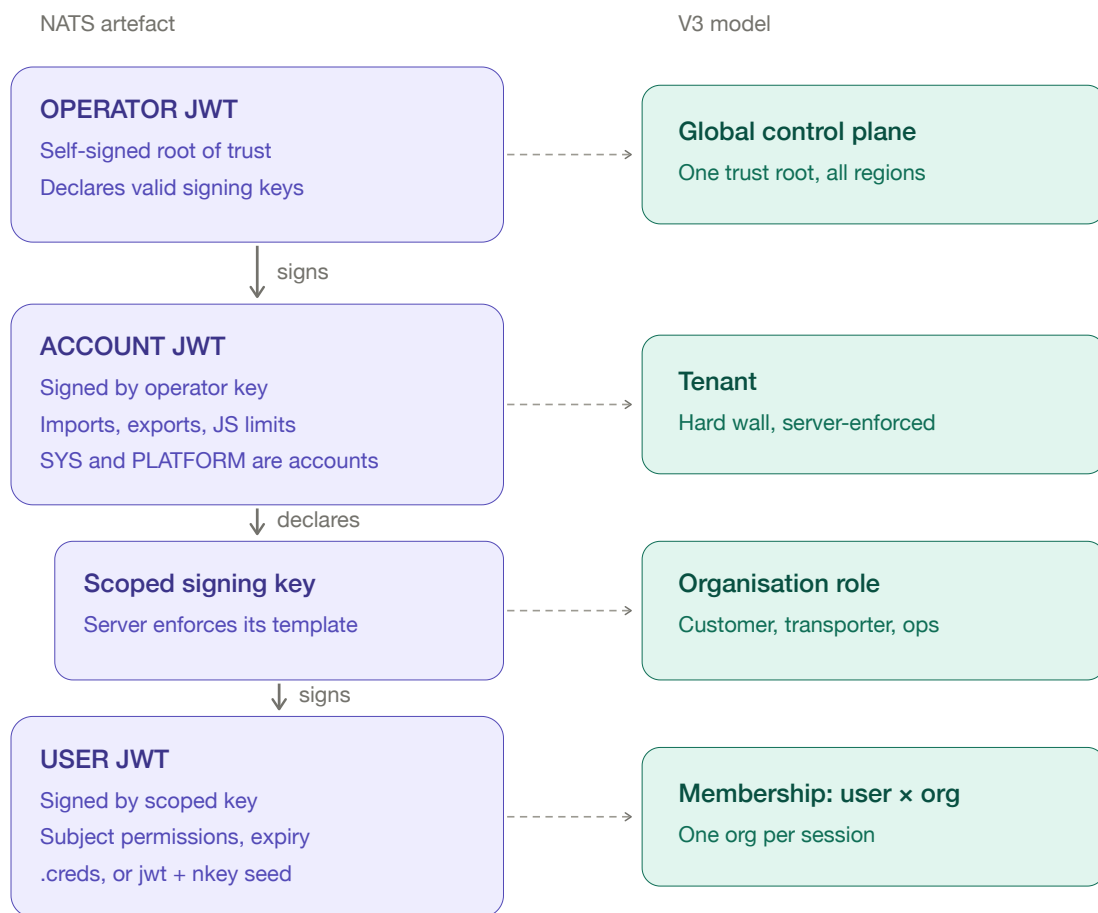


Figure 3 — The operator-mode JWT chain mapped onto the V3 model. The scoped signing key is the missing link that hardens the organisation boundary.

A scoped signing key carries a permission template that the server applies, discarding whatever the user JWT asked for. That is the part of operator mode the earlier diagrams didn't reflect. If `accounts-service` mints a browser credential under a signing key scoped to `PUB api.{ctx}.woolworths.> / SUB org.{ctx}.woolworths.>`, then even a user JWT claiming `>` gets the template — the mint is no longer the thing you are trusting. That moves org isolation from "correct by code discipline in `auth/token.go`" to "correct by server policy," which is a materially different security posture and much closer to the hard wall the account gives you.

It also fixes a specific POC weakness. `Provisioner.CreateUser` currently mints backend users with no permissions set at all — unrestricted within the account. Under a scoped key that becomes structurally impossible rather than something Phase 24b has to remember to fix.

The catch is what goes in the template, and it is a real decision. The template is fixed per signing key, so if it must name a concrete organisation (`woolworths`), you need one scoped signing key *per organisation*, not per role. NATS accounts hold many signing keys happily, but every key lives in the account JWT, and that JWT is re-pushed to the resolver over `$SYS.REQ.CLAIMS.UPDATE` on each change. A tenant with 800 participants means an 800-key account JWT rewritten whenever anyone joins. The alternatives are per-*role* keys with wildcard templates — which cannot separate one customer from another — or accepting the account JWT bloat. Measure it before committing: mint a synthetic account with a few hundred scoped keys and watch push time and resolver directory size.

The ACCOUNT box does double duty. `$SYS` and `PLATFORM` are account JWTs in precisely the same sense a tenant is — same type, same signing chain — but they are platform plumbing, not customers. So "number of accounts" never equals "number of tenants," which is why `accounts-service` reserves both names (BR-AC06) and blocks the `_` prefix (BR-AC07). Any billing or capacity metric derived from account count must exclude them.

One thing worth acting on regardless: the operator signing key currently lives in `accounts-service` as `OPERATOR_SIGNING_KEY_FILE`. In the docs' model that key is the root of trust for every account in every region. A service reachable over HTTP should not hold it. Scoped signing keys are also the mechanism for fixing that properly — delegate account administration to a scoped key and keep the operator identity key offline.

JEREMY

For a logistics app where the control logic (bidding, tenders, load workflows) is the same for all orgs in a region, should that logic form part of the platform (NATS account type) domain — assuming it's the same for all tenants in a region? The downside is that it might need imports and exports between account types. Feedback only.

Should shared workflow live in PLATFORM?

Short answer: no — keep bidding, tenders and load workflow in the **tenant** accounts. But the question conflates two separable things, and once you split them the trade-off mostly dissolves.

Shared logic doesn't require a shared account. "The same for all tenants" is a statement about code, not about where streams live. `shipping-service` already runs one binary holding one `nats.Connect` per tenant account, discovering tenants by scanning the creds directory. That *is* the shared-logic answer: one deployment, one codebase, N connections. Moving the service into PLATFORM doesn't make the logic more shared than it already is — it only changes who owns the data.

What it changes is ownership, and that is the expensive part. Putting load/tender/bid in PLATFORM means every tenant's events sit in one account's JetStream: one resource pool, one set of `JSMem` / `JSMaxFile` / `JSMaxStreams` limits, one blast radius. The per-account limits you provision today become meaningless — a tenant that floods the stream degrades everyone. Isolation reverts to subject-filter correctness in application code, which is precisely what the account boundary was bought to avoid. And a PLATFORM incident stops being "reference data is stale" and becomes "nobody can tender a load."

The migration story bites hardest. Today a tenant that outgrows shared infrastructure — or a white-label operator that leaves — is an account you can move. If their loads are interleaved in a shared PLATFORM stream, extracting five years of event history is an archaeology project. The rule in CLAUDE.md points the same way: whoever owns the stream owns the replay, and tenant history has to be tenant-owned.

The import/export overhead is a symptom rather than the main cost, though worth pricing. Exports and imports live in the account JWTs, so adding an exported subject means re-pushing every tenant's JWT through `$SYS.REQ.CLAIMS.UPDATE` — an $O(\text{tenants})$ operation for something that should be a code deploy. One thing genuinely improves under the export model, in fairness: cross-account service imports give the responder `Nats-Request-Info` identifying the calling account, which is caller identity you don't get within an account. Real, but not enough to outweigh handing PLATFORM ownership of every tenant's state.

The test: does it own state with tenant-specific history? If yes, it belongs in the tenant account. If it is global read-mostly data or a pure function, PLATFORM is right.

Identity, the organisation and tenant registry, reference data, platform config — those export cleanly and are already built that way. Distance calculation, geocoding, document OCR, a rating engine that takes inputs and returns a number would also be fine there. Loads, tenders, bids, trips, POD, payments would not.

The premise deserves pushback too. V2 says the workflow is less uniform than it looks: `PricingType` `{BIDDING, ALLOCATED, FIXED_RATE}` crossed with `LoadType` and `AllocationType`, plus per-business feature flags (`bidding_allowed`, `allocated_allowed`, `allocated_bidding_allowed`, the `*_contract_allowed` set), per-customer fee scales, and bidirectional visibility lists. The *engine* is common; the *configuration* is per-tenant and per-org, and a white-label operator will want to vary it further — that is most of the point of white-labelling. Uniform algorithm plus tenant-specific state and config is the textbook case for shared code over a shared account.

The one thing that would change the answer is cross-tenant matching — a transporter in one tenant taking a load from another. If that is a product goal you need a cross-account path. Even then, export a deliberately narrow service (a matching or capacity-offer contract) rather than relocating the workflow.

Terminology note: NATS has no account types. PLATFORM, `$SYS` and a tenant are all just accounts — the distinction is convention and contents, not anything the server enforces. So there is no structural guardrail stopping business state from drifting into PLATFORM once the first exception is made. That is an argument for drawing the line explicitly in `ARCHITECTURE-ACCOUNTS.md` now, while it is still a one-line rule.

JEREMY

So to clarify further, you're saying that shared logic can reside in a tenant (separation) without hoisting it into the platform layer, because it's just code that's duplicated (close to zero cost or gain)?

Not quite — and the distinction matters, because "duplicated" is doing work in that sentence that it shouldn't.

Nothing about the code is duplicated. It is one repository, one binary, one deployment. What varies is that the process holds N connections — one per tenant account — and instantiates its consumers, streams and KV handles once per connection. That is the pattern `shipping-service` already runs: it scans the creds directory, opens a connection per tenant, and reacts to `notify.accounts.account.*` to add or drop one when a tenant is created or suspended. There is no per-tenant fork, no per-tenant config repo, no code to keep in sync. On the axis you are probably worried about — divergence, maintenance — the cost genuinely is zero.

But the runtime cost isn't zero. What multiplies per tenant is real: a TCP/TLS connection and its goroutines and buffers per service, a set of JetStream streams and consumers, KV buckets, and consumer state the server

holds and checkpoints. Connections are the cheap part. **JetStream stream and consumer count is where you hit a ceiling first**, and it scales linearly with tenants × entity types × read-model shapes.

And the gain isn't zero either. Keeping the workflow in tenant accounts is what makes per-account JetStream limits mean anything — otherwise one tenant's backlog is everyone's problem. It is what makes a tenant extractable when they leave or outgrow shared infrastructure. It is what keeps a subject-filter bug from being a cross-tenant data leak rather than a cross-org one. And per-tenant billing and observability fall out for free instead of needing a `tenantId` predicate on every query.

Consolidating into PLATFORM buys a smaller resource footprint and buys nothing else; keeping it per-tenant costs resources that scale with tenant count and preserves every isolation property you have built. That is a real trade with a crossover point — but it is a *scale* question, not an architecture question, and it can be measured rather than argued.

The standard mitigation, worth knowing before you need it: if per-tenant resources become the constraint, shard tenants across service instances — each instance owns a subset rather than every instance connecting to every tenant. That relieves connection and consumer pressure without touching the account model. Collapsing into PLATFORM should be the last resort, because it is the only one of these options you cannot reverse cheaply.

JEREMY

Would you say that refdata sits in the correct domain (platform) because it sits across all tenants as a read-only store? [and referring also to Figure 1, where the PLATFORM box reads "Identity, registry, refdata"]

Is refdata correctly placed in PLATFORM?

Mostly yes — but both halves of the reasoning deserve challenge, and checking the imports turned up something concrete.

The placement is right for the global corpus. Country codes, currencies, vehicle types, locales, UN/LOCODEs — identical for everyone, read-mostly, and the alternative is replicating the same data into every tenant account and keeping N copies consistent. Exporting it from PLATFORM is exactly what exports are for.

The service side is well built. The four RPC imports in `provisioner.go:194` are scoped per tenant — the remote subject is `rpc.{tenantName}.refdata.item.get.v1` remapped to a context-free local subject — so a tenant can only reach its own namespace, and because the import lives in an operator-signed JWT it cannot remap itself to someone else's. The subject is the caller identity there, which is why refdata-service not reading `Nats-Request-Info` doesn't actually cost anything on the read path.

"Read-only" is the part to push back on. Refdata has draft/publish/rollback and corpus versioning. Something writes it, and more importantly it has *history* — "what was in effect on date X" is a question it can answer. That is the exact example CLAUDE.md uses for reference-looking data that secretly isn't a simple lookup table. It is read-only from a *tenant's* perspective today, which is a statement about who currently has write access, not about the data's nature.

And it doesn't purely "sit across all tenants." The context inheritance tree means `refdata.contexts` carries per-tenant rows, and the `tenant` column there is documented as explicitly *not* a security boundary (BR-D34) because refdata-service runs on shared PLATFORM.

Finding: the stream import doesn't match the care taken on the services

Both sides use an unbounded wildcard in the context position — `nsc add export --account PLATFORM --subject "evt.*.refdata.*.changed" in bootstrap-operator.sh:120`, and `stream("evt.*.refdata.*.changed")` in every tenant's JWT at `provisioner.go:200`. So `acme` receives `refdata` change events for `globex`'s contexts, and vice versa. Not values — but context names, type keys, and change timing, which is enough to infer another tenant's business-unit structure and activity. Four carefully scoped service imports sitting directly above one wildcard stream import reads as an oversight rather than a decision.

The fix follows the pattern already there: narrow the *import* per tenant rather than the export, one entry per context the tenant owns. `accounts-service` already holds `accounts.business_units`, so it knows that set at mint time. The wrinkle is that the subject is keyed by context rather than tenant, so the import list grows with business units instead of being a single line. That is the price of having put context in the subject and tenancy in the account, and it is the right price.

The diagram is hiding the problem in two places

"Identity, registry, refdata" is one box holding three different ownership profiles. Identity and the tenant/organisation registry are genuine control-plane: global, authoritative, read-mostly, no tenant authorship. Refdata isn't one thing — it is a global corpus, plus regional data, plus a per-tenant/per-context overlay, sharing a service and a label.

The single "exports / imports" arrow does the same flattening. It reads as one uniform contract to both tenants. What is actually in the JWT is four tenant-scoped service imports and one wildcard stream import. A diagram that draws the contract as one arrow is part of why the wildcard sat there unremarked.

And there is an inconsistency between Figures 1 and 2. In Figure 1, PLATFORM sits *inside* `Region: za`. In Figure 2 the organisation registry is hoisted into a global control plane above both regions, while PLATFORM — and therefore refdata — stays regional. Both cannot be right:

- If refdata is **regional**, every region holds its own copy of the genuinely global corpus, and you need a publish/replication path to keep them consistent. Workable — read-mostly and slow-changing — but a real mechanism that doesn't exist and isn't drawn.
- If refdata is **global**, it cannot live in a per-region PLATFORM account at all, and the regional stack needs a local cache with a staleness story.

Refdata is most likely **three tiers, not one**: a global corpus that replicates to every region, regional data that never leaves its region (provinces, local port and tax codes), and tenant/BU overlays. Refdata-service's context inheritance tree is already the right shape for the bottom two — `_platform` root, region, tenant, business unit — so the tier may be a property of the context rather than a new concept. What the tree cannot express is cross-region replication of the top tier, because contexts live in a region's Postgres.

JEREMY

What are data candidates for refdata services based on the V2 code implementation?

Refdata candidates from Linebooker V2

Tier 1 — the duplicates, and the ones seeded by hand

V2 has already half-built the reference service for these, and it hurt. Four vocabularies exist *twice*, as both an enum and a table.

V2	Shape	Evidence it wants to be refdata
<code>VehicleTypeEntity</code> + <code>VehicleType</code> enum	~120 enum constants, self parent/child hierarchy, <code>display / fullDisplay / icon</code> , <code>orderIndex</code>	84 seed inserts across <code>AddNewTruckTypes.sql</code> , <code>AddCarCarrierTruckType.sql</code> , <code>AddNewMoffettVehicleType.sql</code> , <code>AddRackedVolumax.sql</code> , <code>SetVehicleTypeImages.sql</code>
<code>DocumentTypeEntity</code> + <code>DocumentTypes</code> enum	22 constants, <code>documentClass</code> long-term/transactional	8 inserts across 7 separate migrate scripts
<code>CommodityCategoryEntity</code> + <code>CommodityCategory</code> enum	13 constants + display	<code>MigrateCommodityCategories.sql</code>
<code>LoadUnitTypeEntity</code> + <code>LoadUnitType</code> enum	7 constants; table has no FK at all — pure picklist served over REST	<code>AddLiquidLoadUnit.sql</code>
<code>JobTitleEntity</code>	<code>title</code> + <code>isDefault</code>	42 seed inserts

The tell isn't the duplication itself — it is that adding a truck type requires a migrate script and a deploy. That has happened at least seven times for vehicle types alone.

`VehicleTypeEntity` is the one to single out, because it carries a `customer_profile_entity_id` FK. A global truck-type catalogue that is simultaneously per-customer, in the same table, with no separation. That is exactly the context-inheritance tree refdata-service already implements — global root, tenant overlay, BU overlay — and it is the strongest validation that the POC's context model solves a real V2 problem rather than an invented one.

Tier 2 — picklists still trapped as enums

Controlled vocabulary a business user would plausibly want to extend without a deploy: `DeclineReasonType`, `PodRejectionReason`, `FailType`, `AdditionalCostType` (16, with display strings like "Standing time", "Weigh bridge fine", "Chep Cost"), `AdditionalCostTransactionType` (9), `SpecialInstructionType` (corner plates, straps, tarps, PPE...), `TrackingProviderNames` (36 telematics providers), `BusinessEntityType`, `ReferralSource`, `TemperatureType`, `VatRate`, `EstimationPeriodType`, `LoadAlertType`, `TransporterProviderCount`, `MatchCriterion`.

`PolicyChangeEnum` deserves its own mention — a single constant, `PRIVACY_POLICY_JULY_TWENTY_ONE`, with a backing table carrying `effective_date`. Pure data wearing an enum costume, and the only non-pricing lookup in V2 with temporal semantics.

Geography — with one important caveat

V2 has two competing hierarchies that were never linked: the legacy `CountryEntity` → `RegionEntity` → `TownEntity` (and `CountryEntity` has **no ISO code column at all** — just a name), and the modern `GeoAreaEntity` (`geo_areas`, string PK, `level` COUNTRY/REGION/MUNICIPALITY, `countryCode`,

`parentId`, `PostGIS` `center` + `geom`). `MoveWesternProvinceToZambia.sql` illustrates what maintaining the legacy one costs.

Split the hierarchy from the geometry. Names, ISO codes, parent relationships and level are `refdata` — cacheable, small, exactly what a KV-backed dictionary is for. `MultiPolygon` boundaries are megabytes per row and belong in a spatial service with `PostGIS`, not in a corpus replicated into every region's KV. Pushing geometry through `refdata`'s cache would be the first thing to blow up.

Tiering falls out naturally: ISO countries and currencies are global; provinces, municipalities, local port and tax codes are regional; nothing here is tenant-scoped.

Config — candidate, but draw a line

`ConfigEntity` is a flat global key/value table (`config.maxFeePercentage`, `config.script_lock_timeout`, `config.whatsapp.from`, `config.fee-scale.default.id.load`) **with no business dimension**. The consequence is visible: per-tenant settings leaked into bespoke tables like `PodNotificationConfigEntity`, seeded one customer at a time by `MigrateSFTPConfigForPepsi.sql`. Context-scoped config in `refdata` fixes that class of problem directly.

But `PodNotificationConfigEntity` also holds SFTP passwords. **Secrets don't go in `refdata`**, and especially not in `refdata-as-it-is` — shared PLATFORM account, `tenant` column documented as not a security boundary. Config splits into non-sensitive settings (`refdata`) and credentials (tenant-owned, encrypted, separate store).

What `refdata` brings that V2 doesn't have

Versioning. No pure lookup in V2 is versioned or effective-dated — corrections are destructive migrate SQL. Draft/publish/rollback is a genuine upgrade, and there are no existing temporal semantics to preserve, so the design is free.

Localization, which V2 doesn't need — yet. There are no locale bundles, no translation tables, no `locale` column anywhere. Display text is single-language, baked into `display / fullDisplay` columns or enum constructors. What *does* exist is `messages.properties` versus `messages-thornlands.properties` — white-label brand wording, not language. So `refdata`'s localization feature will probably find its first real customer as **per-tenant label overrides for white-labelling**, not translation. Same mechanism, different framing, and a better argument for the feature than i18n is.

Explicit non-candidates

The ~90 state-machine enums stay in code: `LoadStatusType`, `LoadProgressStatus` (20 composite states), `LoadHistoryType` (38), `LoadEventSnapshotActionType` (53), `BidStatusType`, `BidType`, `PricingType`, `AllocationType`, `TenderStatus`, `PodStatusType`, all the `*SortByType`. The test: the code switches on these, so making them data means a business user could add a status no handler knows about. `ReportType` (embedded URLs + Metabase role maps) is config-in-code that should be extracted, but into deployment config, not a dictionary.

Borderline, worth a decision rather than a default: `EmailType` (60+) and `NotificationTemplate` (24). The *catalogue* is vocabulary, but each constant maps to a resource file. Given the Thornlands white-label precedent, template content per tenant is a decent `refdata` candidate — but a bigger conversation than picklists.

Open decisions

1. **Tenant = marketplace-operating business, or country?** The white-label pipeline argues for the former; the commercial roadmap decides.
2. **Organisation isolation inside a tenant account — day one, or deferred?** Scoped signing keys make it server-enforced; measure the account-JWT size cost first.
3. **Participant-plane subject grammar** — arity-7 `api.*`, or a separate `org.*` family (preferred).
4. **Is reldata global or regional?** Figures 1 and 2 currently answer this both ways. Settle it before `Multi-Region-Plan.md` comes off DRAFT.
5. **Naming** — retire "Operator" for the business entity; free "Region" from V2's province meaning; pick one tenant-name convention before seeding.

Generated from the working session. Diagrams are reproductions of the inline SVGs shown during the conversation. Code references point at `demos/01-dictionary/` in `nats-tech-lab` and at the Linebooker V2 repository.